XDA003 Belt / Hook 參數設定說明 v3

> 適用版本：Belt v_new（2026-05-09 起的韌體）+ Hook v_new
>
> v3 與 v2 的差異：把原本保留的 `advertising_duration` 欄位重新定義為 `flags` 旗標位元組，新增 `HOOK_SETTINGS_FLAG_IN_SAFE_ZONE` 與「flag-only write」哨兵語意，用於支援 SAFE-zone 通知與旁路通道。Wire ABI 與 v2 完全相容（payload size、欄位偏移皆未動），既有後台不需更動也能繼續推 v2 設定，只是無法觸發 v3 新增的副作用。

---

1. 用途與通道

本協定描述後台如何透過 LoRaWAN downlink fPort=30 設定 Belt 與其綁定的 Hook 參數。Belt 收到 fPort=30 packet 後會：

1. 依 payload 長度決定解析方式：
   - 24 bytes — 只更新 Belt 自身參數
   - 74 bytes — 更新 Belt 自身參數 加 至多 2 支 Hook 的參數
2. Hook 參數區塊會比對 `id[6]`，只有與當下綁定 Hook ID 相符的區塊才會被 Belt 透過 BLE 轉送到對應 Hook；不相符的區塊會被靜默丟棄。

> 嚴格長度檢查：非 24 / 74 byte 的 fPort=30 packet 必須被 Belt 韌體忽略；不允許部分解析。

---

2. Wire 結構

2.1 Belt header（24 bytes）

```c
struct PACKED belt_downlink_header_v2
{
uint8_t  reboot_and_ignore_reed;  // 1
uint8_t  reset;                   // 1
uint16_t belt_sleep_duration;     // 2
uint16_t uplink_report_interval;  // 2
uint8_t  beacon_scan_duration;    // 1
uint8_t  beacon_scan_times;       // 1
char     beacon_name_filter[8];   // 8
char     hook_name_filter[8];     // 8
};
```

欄位	大小	說明

`reboot_and_ignore_reed`	1	複合控制：`1` = 立即重開機（最高優先）；`200` / `201` = 不重開機，停用 / 啟用磁簧（v_new 已棄用磁簧偵測但仍接受指令）；`0` = 無動作
`reset`	1	重設旗標。保留位元；本欄位非 0時依後台版本另行定義
`belt_sleep_duration`	2	Belt 一般作業循環中的休眠秒數（little-endian）
`uplink_report_interval`	2	一般狀態定時上傳間隔（秒，little-endian），1–3600
`beacon_scan_duration`	1	每次 BLE beacon 掃描秒數
`beacon_scan_times`	1	每個循環的掃描次數
`beacon_name_filter`	8	beacon 名稱前綴過濾（預設 `abeacon`）
`hook_name_filter`	8	Hook 名稱前綴過濾（預設 `SQ`，v_new 改 `HOOK-` 但仍接受 `SQ` 以相容）

2.2 Hook 區塊（25 bytes，最多 2 支）

```c
struct PACKED hook_downlink_command_struct_v2
{
uint8_t  id[6];
uint16_t reserve;
hook_settings_struct_v2_t hook_settings;
};                          // 25 bytes
```

欄位	大小	說明
`id[6]`	6	Hook 的 base MAC（MSB-first）。Belt 會同時嘗試 MSB-first 與 LSB-first 比對綁定，所以後台兩種順序皆可接受
`reserve`	2	保留欄位，目前全填 `0x0000`
`hook_settings`	17	內嵌的 hook 設定結構，見 §2.3

2.3 hook_settings_struct_v2_t（17 bytes）— v3 變更

```c
typedef struct PACKED hook_sensing_axis_struct
{
uint8_t  sensing_type;       // 0 = EXCLUSIVE, 1 = INCLUSIVE
uint16_t sensitivity_high;
uint16_t sensitivity_low;
} hook_sensing_axis_struct_t;    // 5 bytes
typedef struct PACKED hook_settings_struct_v2
{
uint8_t alarm_window_s;   // 1–30秒，0 為哨兵值（見 §3）
uint8_t flags;            // ← v3 重新定義（v2 為 advertising_duration）
hook_sensing_axis_struct_t x;
hook_sensing_axis_struct_t y;
hook_sensing_axis_struct_t z;
```

} hook_settings_struct_v2_t; // 17 bytes

```

| 欄位 | 大小 | 說明 |
|---|---|---|
| `alarm_window_s` | 1 | Hook alarm window（秒）。值範圍 1–30，超過或為 0 會被 Hook 端 clamp。值 0 在 v3 起賦予新語意（哨兵），見 §3 |
| `flags` | 1 | v3 起為旗標位元組（v2 為 `advertising_duration`，已棄用）。各 bit 定義見 §3。transient 不持久化 — Hook 收到後執行 side-effect，存進 flash 時固定為 0 |
| `x` / `y` / `z` | 各 5 | 三軸磁場感測參數：`sensing_type` + `sensitivity_high` + `sensitivity_low`（uint16 little-endian） |

> v2 → v3 ABI 相容：`flags` 與 `advertising_duration` 同位元組同寬度。舊版後台若在這個 byte 填 0、1、2 等保留值，bit `0x80` 不會被誤觸發，行為等同 v2。

### 2.4 完整 74-byte payload

```c
struct PACKED belt_downlink_command_struct_v2
{
belt_downlink_header_v2 header;        // 24 bytes
hook_downlink_command_struct_v2 hook[2]; // 50 bytes
};
```

`hook[0]` 與 `hook[1]` 各自獨立比對綁定 ID，可單獨設定；不需要的 slot 把 `id[6]` 全填 `0x00` 即會被忽略。

---

3. v3 新增：`flags` 與 flag-only write

3.1 旗標位元定義

```c
#define HOOK_SETTINGS_FLAG_IN_SAFE_ZONE  0x80   // 作業員位於 SAFE zone，請 pin 住 counter
// 其餘 bit 保留，必須填 0
```

Bit	名稱	行為
`0x80`	`IN_SAFE_ZONE`	Hook 收到後設定 SAFE-zone grace window（10秒），期間 mag task 強制將 `not_detected_counter` 與 `status` 釘在 0；同時刷新內部 `last_metal_seen_ms`，讓 grace 過期時也是從 0 開始重新計時
`0x01–0x40`	保留	必須填 0；Hook 端會忽略

3.2 Flag-only write（`alarm_window_s = 0` 哨兵）

`alarm_window_s` 原本範圍是 1–30；當值為 0時 Hook v3 將其視為「flag-only write」哨兵：

- Hook 不會 把 17 byte 寫進 settings flash（XYZ 軸與 alarm_window_s 全部忽略）
- Hook 僅 處理 `flags` 欄位的副作用
- 仍會回傳一筆 ACK（`0x00 + 17B 當下實際 settings`）讓送方確認 hook 還活著；ACK 中的 settings 維持原值

這允許 Belt（或後台）在 不知道 Hook 當下完整 settings 的情況下單純通知一個 side-effect，不必先讀回設定再原樣回寫。
```

3.3 Belt 的自動 SAFE-zone 觸發

Belt v_new 韌體在「看到任一 SAFE-role beacon 為 fresh」期間，會 自動 透過本機 BLE 對所有已連線的 Hook slot 每 3秒送一次 flag-only write（`alarm_window_s=0`、`flags=0x80`），維持 Hook 端的 grace window：

```
SAFE 進入：      [flag] [flag] [flag] [flag] ... 每 3s
 |
 ▼
Hook grace window: ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓
 | 10s | 10s | 10s | ...
每收到 flag 就刷新
SAFE 離開：      (停止送 flag)
 ▼
Hook grace window: ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓ ← 最後一次 flag 後 10s 自動失效
```

這個流程 不需要後台介入：只要場域 SAFE beacon 部署正確，Belt 就會自己處理。

3.4 後台手動觸發（可選）

如需後台主動 reset 某支 Hook 的 counter（例如測試或排程性「重置作業日」），可以透過 fPort=30 推一筆 74-byte payload，其中對應的 hook 區塊：

```
id[6]:              目標 Hook 的 base MAC
reserve:            0x0000
hook_settings:
alarm_window_s:   0x00        ← 哨兵
flags:            0x80        ← FLAG_IN_SAFE_ZONE
x / y / z:        (忽略，建議全填 0)
```

Belt 收到後會比對 ID、轉送這個 17-byte block 到對應 Hook 的 BLE RX；Hook 看到 `alarm_window_s=0` 會走 flag-only path，不更動 settings、僅執行 reset counter。

---

4. 範例 payload

4.1 純更新 Belt 設定（24 bytes）

```
01 00 00 1E 00 3C 04 02 61 62 65 61 63 6F 6E 00 53 51 00 00 00 00 00 00
 |  |  └────┘  └────┘  |  |  └──────────────"abeacon"──────────────┘  └──────────"SQ"──────────┘
 |  |   |    |    |  └── beacon_scan_times = 2
 |  |   |    |  └──────── beacon_scan_duration = 4 s
```

```
│   │   │   └── uplink_report_interval = 60 s
│   │   └── belt_sleep_duration = 30 s
│   └── reset = 0
└── reboot_and_ignore_reed = 1（重開機）
```

4.2 設定 Belt + 一支 Hook 的感測參數（74 bytes）

```
header (24B):  00 00 00 1E 00 3C 04 02 61 62 65 61 63 6F 6E 00 53 51 00 00 00 00 00 00
hook[0] (25B): DC 06 75 F8 2C C8 00 00 0A 00 01 64 00 32 00 01 64 00 32 00 01 64 00 32 00
└──────── id ──────────┘ └rsv┘ │   │   └──────────────────────── x / y / z 各 5 byte ────────────────────────┘
│   └── flags = 0（不觸發任何 side-effect）
└──────── alarm_window_s = 10 s
hook[1] (25B): 00 00 00 00 00 00 00 00 ... (id 全 0 → Belt 忽略)
```

4.3 後台手動 reset hook[0] 的 counter（74 bytes）

```
header (24B):  00 00 00 1E 00 3C 04 02 ...（沿用既有設定即可）
hook[0] (25B): DC 06 75 F8 2C C8 00 00 00 80 00 00 00 00 00 00 00 00 00 00 00 00 00 00
└──────── id ──────────┘ └rsv┘ │   │
│   └── flags = 0x80 (IN_SAFE_ZONE)
└──────── alarm_window_s = 0（flag-only 哨兵）
hook[1] (25B): 00 00 00 00 00 00 00 00 ...
```

---

5. 韌體側驗證 / clamp

條件	Belt 端行為	Hook 端行為
payload 長度 ≠ 24 / 74	整包丟棄	—
Hook ID 不在綁定列表	該 hook block 不轉送	—
`alarm_window_s = 0`	直接轉送 17 bytes	觸發 flag-only path，不 persist settings
`alarm_window_s > 30`	直接轉送	clamp 回 `DEFAULT_HOOK_CYCLE_TIME / 1000`（10秒）
`flags & 0x80` 為 1	—	設定 SAFE-zone grace window 10秒
`flags & 0x7F` 非 0	—	忽略（保留位元）
任何寫入到 flash 之前	—	`flags` 強制歸 0（transient 不持久化）

---

6. 向後相容矩陣

後台版本	Belt 韌體版本	Hook 韌體版本	行為
v2	v_new	v_new	OK；`flags` 在 v2 後台填的是舊 `advertising_duration` 值（通常 1 或 2），不會觸發 `0x80` bit
v3	v_new	v_new	OK；完整支援 SAFE-zone reset 與 flag-only write
v3	v_new	舊版	不安全 — 舊 Hook 看到 `alarm_window_s=0` 不會走 flag-only path，會把 settings 全打成 0（XYZ sensitivity 都 0 → 偵測失效）。必須先把 Hook 升到 v_new
v2	舊版	v_new	OK；Belt 把 17 bytes 原樣轉送，Hook 仍可解析

升級順序建議：先燒 Hook → 再燒 Belt → 最後切換後台到 v3 推 payload。

7. 相關文件

- `docs/belt_operation.md` — Belt 端使用者操作說明（含 SAFE / DANGER zone 行為描述）
- `docs/hook_operation.md` — Hook 端使用者操作說明
- `firmware/src/data_types.h` — Belt 端 wire struct 權威定義
- `22_XDA003H/firmware/library/data_types.h` — Hook 端 wire struct 權威定義